Crewly UI and Pricing Strategy Proposal

Crewly UI Consistency and Pricing Strategy Proposal

Prepared by: Mia (Product Manager, crewly-product-mia-member-1)
Collaborators: Ava (UX Designer), Sam (Tech Lead), Victor (AI CEO, Pro)
Date: 2026-03-27

---

1. Executive Summary

This proposal outlines a coordinated plan to improve the consistency of the Crewly OSS user interface and resolve concerns regarding pricing information accuracy in local installations. We recommend a phased extraction of shared UI components and a shift from hardcoded to dynamic, API-driven pricing definitions.

---

2. UI Component Reuse Evaluation

Based on the audit by Ava and Sam.

2.1 Current State

Our audit revealed significant inconsistencies across the application’ s core pages: - **Chat & Security**: These pages currently use **zero** components from our shared UI library, relying entirely on custom inline styles and ad-hoc implementations. - **Settings**: Uses the UI library partially but still contains over 25 instances of duplicated error banners, loading spinners, and section patterns. - **Library Adoption**: 53% of our 38 core components bypass the UI library entirely.

2.2 Recommendation: Shared UI Library Extraction

We propose extracting 6 new shared components to eliminate 80% of current duplication: 1. **AlertBanner**: Replaces 10+ inline error/success messages. 2. **LoadingSpinner**: Standardizes 8+ different loading states. 3. **SettingsSection**: Replaces 8+ custom section containers. 4. **ConnectionDetailsCard**: Consolidates 6 messenger integration layouts. 5. **ExpandableSection**: Standardizes UI patterns in Security and Settings. 6. **MetricCard**: Unifies statistical displays across Dashboard and Security.

2.3 Implementation Roadmap

- **Sprint A (Quick Wins)**: Extract Tier 1 components (Banner, Spinner, Section) and replace 25+ duplicates.
- **Sprint B (Chat Integration)**: Refactor Chat components to use `Button`, `Badge`, and `Alert`.
- **Sprint C (Security Standardization)**: Standardize status indicators and card layouts.
- **Sprint D (Advanced Extractions)**: Implement `DataTable` and `MetricCard`.

---

3. Pricing Strategy Analysis

Addressing user concerns regarding stale pricing in local OSS installations.

3.1 The Problem

Currently, pricing for 'Solo' and 'Team' tiers is hardcoded in the OSS frontend (`Pricing.tsx` and `payment-wall.types.ts`). If prices are updated on the main site, local installations will display incorrect information, leading to user confusion and trust issues.

3.2 Proposed Strategy: Dynamic Pricing

1. **Zero Hardcoding**: Remove all static price values from the OSS codebase.
2. **API-Driven Fetching**: The frontend will fetch pricing plans from a Cloud API (`api.crewlyai.com/v1/pricing`) on demand.

3. **Intelligent Fallbacks**:
    - **Offline Mode**: Show a generic "Premium features available" message with a link to the live pricing page.
    - **Connected Mode**: Show live, accurate pricing based on the current user's region and currency.
4. **Contextual Visibility**: Only show detailed pricing tables when the user initiates an "Upgrade" flow or connects their Cloud account.

---

4. Strategic Vision: OSS vs. Pro

Integrating the perspective of Crewly Pro (Victor).

- **OSS (The Engine)**: Remains free and open-source, focusing on core agent management and P2P collaboration. It serves as the primary funnel for the ecosystem.
- **Cloud/Pro (The Service)**: Provides managed infrastructure, centralized memory, and premium integrations.
- **UI Distinction**: The OSS UI should be optimized for **Utility and Control**, while the Cloud UI handles **Billing, Growth, and Collaboration**.

---

5. Conclusion

By unifying our UI components and making our pricing data dynamic, we create a more professional, maintainable, and trust-worthy product. This strategy allows us to iterate on our business model without requiring users to update their local installations to see accurate information.

Next Steps: 1. Approve the 4-sprint UI roadmap. 2. Implement the `/pricing/plans` API endpoint on the Cloud backend. 3. Refactor `frontend/src/pages/Pricing.tsx` to use the new dynamic fetch.